

Customized Virtual File System (CVFS)

Project Overview:

This Project is custom implementation of a **Virtual File System (VFS)** that simulates the core functionality of the Linux file System. It is built entirely in C, with its own custom shell to interact with the virtual environment. The Project provides a practical understanding of **System Calls, File Handling, memory management, and OS internals**.

Technology Used:

Component	Technology
Programming Language	C
Operating System	Linux
Compiler	GCC
Editor	Visual Studio Code
Terminal	Linux Shell
Libraries Used	stdio.h, stdlib.h, string.h, unistd.h

Key Features:

- **Custom Shell Interface**
 - Provides Linux-like commands for file operations (create, open, close, read, write, unlink, ls, display, etc.)
- **System Call Simulation**
 - Implements core Linux file system calls (open, read, write, unlink, close, create, etc.) using C.
- **File System Data Structures**
 - Incore Inode Table(IIT)
 - File Table(FT)
 - UAREA(User Area)
 - User File Descriptor Table(UFDT)
- **Memory Management.**
- **Platform Independent**
 - Allows system-level file handling functionalities of Linux to be used on any operating system platform.
- **Database-like Functionality**
 - Provides a **customized database management layer** with structured file handling.

User Interface Used:

The project uses **Command Line Interface (CLI)**
The user interacts through Linux Terminal.

Platform Required:

Ubuntu Linux
Windows with WSL (Optional)
Any Linux Distribution

Hardware Requirements:

Minimum Intel i3 Processor
4 GB RAM 100 MB Free Disk Space
Recommended Intel i5/i7 8 GB RAM
Ubuntu Linux

Learning Outcomes:

- Deep Understanding of **Linux File System internals**.
- Practical knowledge of data structures used in OS -
 - Inode Management
 - Super Block
 - File Tables
 - User File Descriptor Table (UFDT)
 - UAREA
- Strong grasp of **System programming in C**.
- Hands-on with **shell design & command interpreter**.
- Application of **low-level logic building** for OS-like environments.

GitHub Repository:

[https://github.com/codesbyshital/Programming/tree/main/C Programming/Projects/CVFS](https://github.com/codesbyshital/Programming/tree/main/C%20Programming/Projects/CVFS)

Author:

Shital Ajit Nikam

Data structures used in the project:

The Customized Virtual File System (CVFS) is completely memory-based. To simulate the behavior of an operating system's file system, several interconnected data structures are used. These structures maintain information about files, users, open files, storage management, and metadata.

The major data structures used in the project are:

1. SUPERBLOCK
2. INODE
3. FILETABLE
4. USER AREA (UFDT)
5. BOOT BLOCK
6. Linked List of Inodes

Boot Block:

The Boot Block stores the information related to the file system itself. It is initialized when the Virtual File System starts.

Although it does not directly participate in file operations, it stores basic file system information.

```
struct BootBlock
{
    char Information[100];
};
```

Description:

Member	Description
Information	Stores information related to the Virtual File System.

Super Block:

The Super Block is the control block of the Virtual File System. It maintains the status of available and occupied inodes.

Whenever a file is created or deleted, the Super Block is updated.

```
struct SuperBlock
{
    int Totallnodes;
    int Freenodes;
};
```

Description:

Member	Description
TotalInodes	Total number of inodes created during initialization.
FreeInodes	Number of currently available free inodes.

Advantages:

- Tracks free storage.
- Prevents creation of files when no inode is available.
- Speeds up allocation.

Inode:

The inode is the most important data structure in the project.

Every file in the Virtual File System is represented by exactly one inode.

An inode stores metadata about a file but does not directly store the filename's contents.

Instead, it stores a pointer to the file's data buffer.

struct Inode

```
{  
    char FileName[20];  
    int InodeNumber;  
    int FileSize;  
    int ActualFileSize;  
    int FileType;  
    int ReferenceCount;  
    int Permission;  
    char *Buffer;  
    struct Inode *next;  
};
```

Member	Description
FileName	Name of the file
InodeNumber	Unique inode identifier
FileSize	Maximum allocated size
FileActualSize	Current size of data written
FileType	Regular or Special file
Buffer	Pointer to file contents

Member	Description
LinkCount	Number of file links
ReferenceCount	Number of active file descriptors
Permission	Read, Write or Read-Write permission
next	Pointer to next inode

Role:

- Represents one virtual file.
- Stores all metadata.
- Connected through a linked list.

FILETABLE:

Whenever a file is opened, a new File Table entry is created. It stores information specific to that open instance of the file.

```
struct FileTable
{
    int ReadOffset;
    int WriteOffset;
    int Mode;
    PINODE ptrinode;
};
```

Member Description:

Member	Description
ReadOffset	Current read position
WriteOffset	Current write position
Count	Number of users using this file descriptor
Mode	Read / Write / Read-Write mode
ptrinode	Pointer to corresponding inode

A file can be opened multiple times.

Each open operation creates a different File Table. Each file descriptor maintains its own offsets independently.

Example:

Demo.txt

FD 3 ---> FILETABLE 1
FD 4 ---> FILETABLE 2
FD 5 ---> FILETABLE 3

UFDT (User File Descriptor Table):

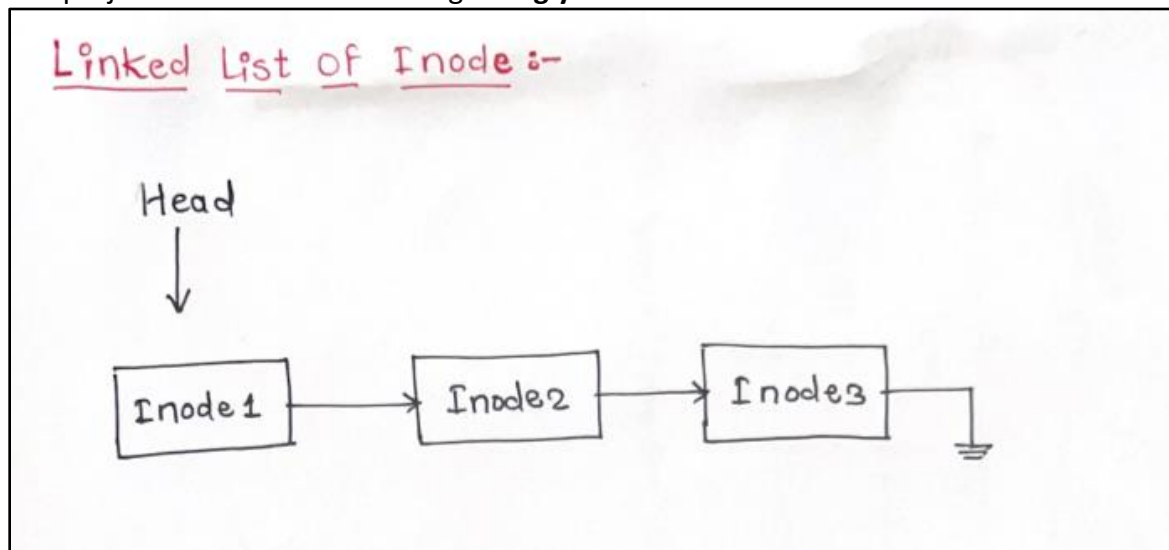
UFDT stores all currently opened file descriptors.
Each entry points to a File Table.

```
struct UAREA
{
    char ProcessName[20];
    PFILETABLE UFDT[MAXOPENFILES];    // how many files gets opened
};
```

Member	Description
UFDT	Array of pointers to File Tables

Linked List of Inodes:

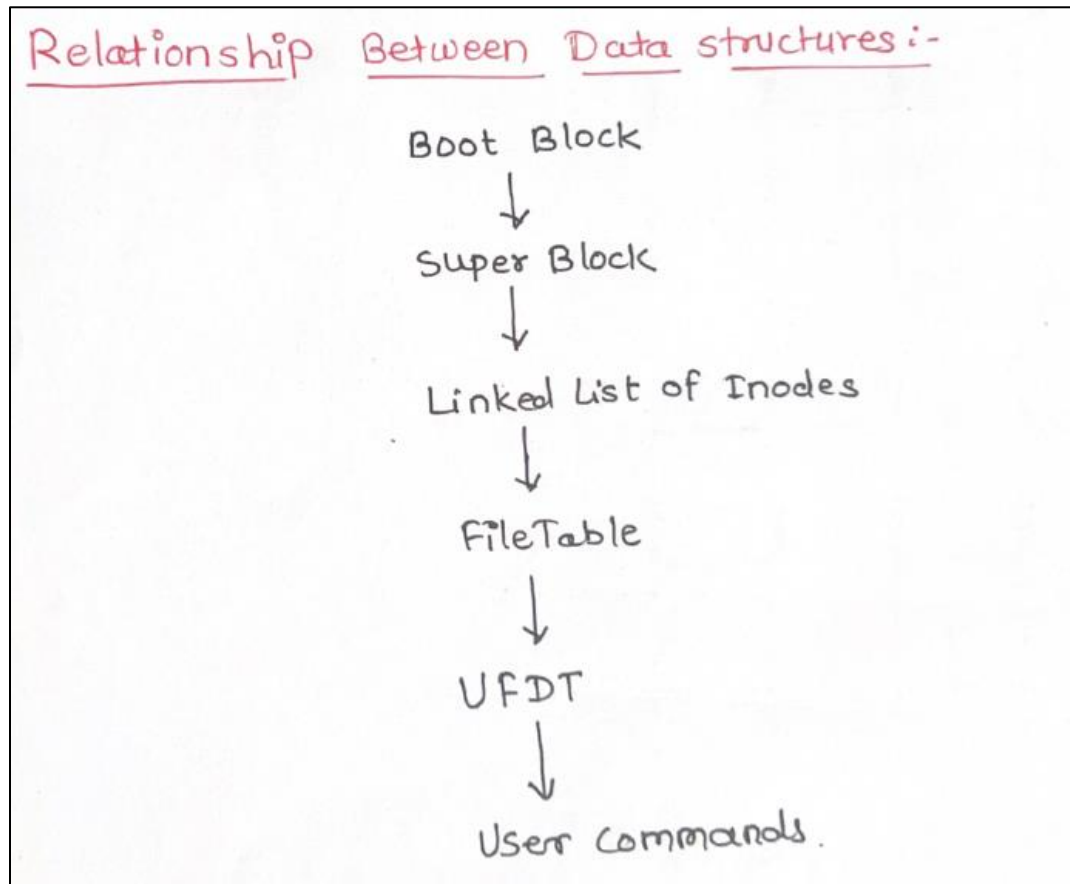
The project maintains all files using a **Singly Linked List**.



Advantages:

- Dynamic allocation.
- Easy insertion.
- Efficient traversal.
- Memory efficient.

Relationship between Data Structures:



Data Structure Interaction:

Suppose the user executes: create Demo.txt 3

The following steps occur:

- Super Block checks whether a free inode is available.
- A free inode is allocated.
- Memory buffer is allocated.
- A File Table entry is created.
- UFDT stores the File Table pointer.
- File Descriptor is returned to the user.

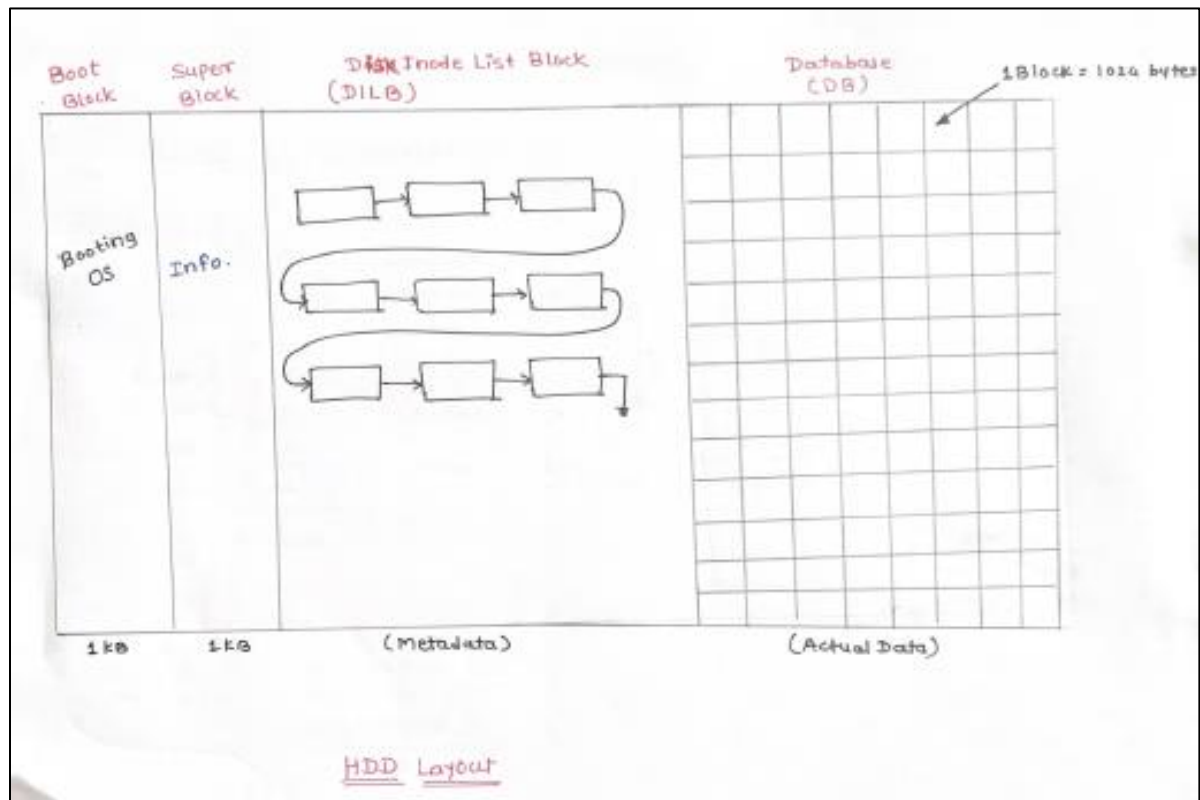
Similarly, when a file is opened: open Demo.txt 3

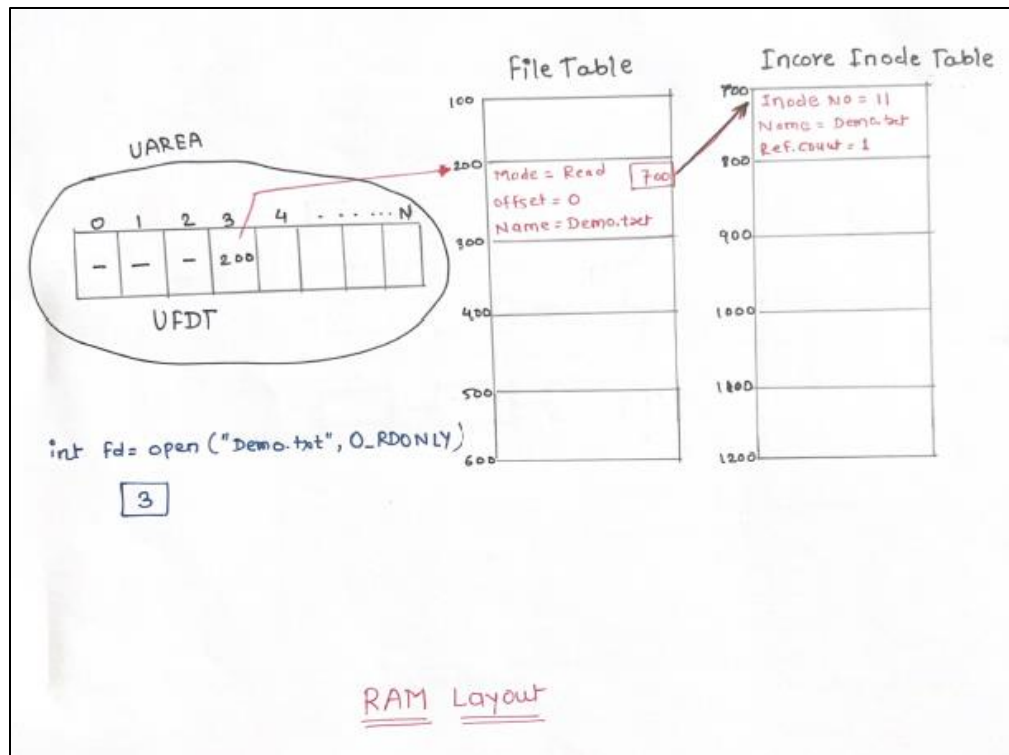
A **new File Table** is created, while the existing inode is reused. This allows multiple file descriptors to refer to the same file while maintaining independent read and write offsets.

Importance of these Data Structures:

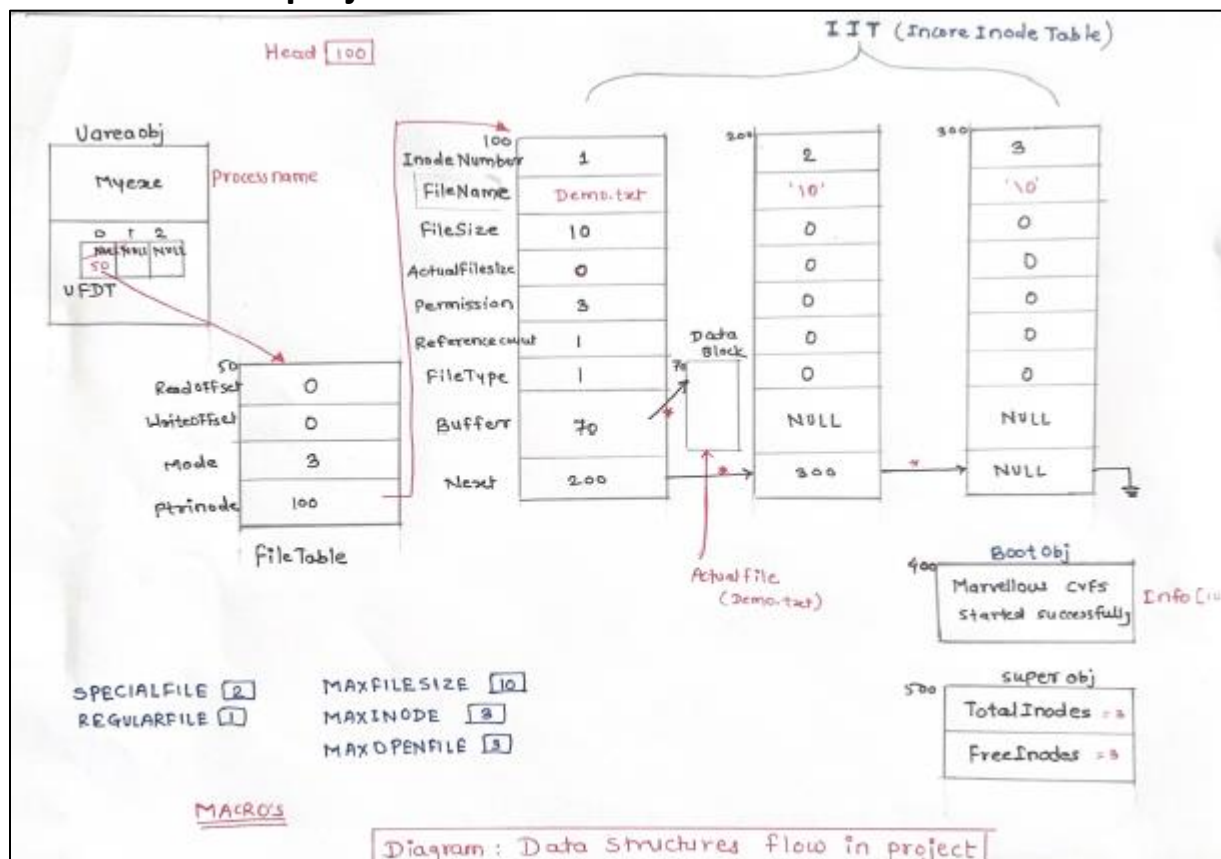
Data Structure	Reason for Selection
Super Block	Efficient management of free and occupied inodes
Inode	Stores all file metadata independently of file contents
File Table	Maintains per-open-file information such as offsets and mode
UFDT	Provides quick access to open files through file descriptors
Linked List	Dynamic storage and traversal of inodes without fixed array limitations
Boot Block	Stores information related to the virtual file system initialization

Diagram of data structures used in the Project :





The flow of the project:



Actual code of the project:

<..\..\Git\C Programming\Projects\CVFS\CVFS.c>

MACRO's used the in Project:

User Defined Macros:

```
#define MAXINODE 10
#define MAXFILESIZE 1024
#define MAXOPENFILES 10
```

```
#define READ 1
#define WRITE 2
#define EXECUTE 4
```

```
#define START 0
#define CURRENT 1
#define END 2
```

```
#define EXECUTE_SUCCESS 0
```

```
#define REGULARFILE 1    //used this macro for type of file
#define SPECIALFILE 2
```

```
#define BACKUP_PATH "C:\\Users\\ADMIN\\Desktop\\LB\\CVFS\\Backup"
```

User Defined Macros for error handling:

```
#define ERR_INVALID_PARAMETER -1
```

```
#define ERR_NO_INODES -2
```

```
#define ERR_FILE_ALREADY_EXIST -3
```

```
#define ERR_FILE_NOT_EXIST -4
```

```
#define ERR_PERMISSION_DENIED -5
```

```
#define ERR_INSUFFICIENT_SPACE -6
```

```
#define ERR_INSUFFICIENT_DATA -7
```

```
#define ERR_MAX_FILES_OPEN -8
```

```
#define ERR_FILE_NOT_OPEN -9
```

Functions Used in the Project:

1. Function Name : **InitialiseUAREA()**
Input Parameters: NA
Return Value: print Message as "UAREA gets initalised successfully"
Description: It is used to initialize UAREA
2. Function Name : **InitialiseSuperBlock()**
Input Parametres: NA
Return Value: print Message as "Super Block gets initialized successfully"
Description: It is used to initialize SuperBlock
3. Function Name : **CreateDILB()**
Input Parameters: NA
Return Value: print Message as "DILB gets created succesfully"
Description: It is used to create linked list of inodes & Initialize its members.
4. Function Name : **StartAuxillaryDataInitialisation()**
Input Parametres: NA
Return Value: print Message as "Bootng process of Marvellous CVFS is completed"
Description: It is used to call all such functions which are used to initialize Auxiliary data. This is dummy function like OS boot process.
5. Function Name : **DisplayHelp()**
Input Parameters: NA
Return Value: Displays command & its usage for user.
Description: It is used to Display help to the user of project.
6. Function Name : **ManPageDisplay()**
Input Parameters: NA
Return Value: Displays manpage
Description: It is used to Display manpage of specific command
7. Function Name : **IsFileExist()**
Input Parameters: Name of file
Return Value: True if present, false if not present
Description: It is used to check whether the file exist or not.
8. Function Name : **CreateFile()**
Input Parameters: Name of file, Permissions
Return Value: File Descriptor
Description: It is used to create a new virtual file in the file system with the specified name and permission.

Permission Values:

- 1 – Read
- 2 – Write
- 3 – Read & Write

Operation Performed:

- Checks for free inode.
- Verifies whether the file already exists.
- Allocates memory for the file buffer.
- Creates a File Table entry.
- Updates the Super Block.

9. Function Name : **LsFile()**

Input Parameters: NA

Return Value: List of Files available.

Description: Displays the names of all existing files.

Operation Performed:

- Traverses the inode list.
- Displays all files having a valid file type.

10. Function Name : **LsFile_All()**

Input Parameters: NA

Return Value: List of all files with complete information.

Description: Displays complete information about all files present in the Virtual File System.

Information Displayed:

- File Name.
- Inode Number.
- Actual File size
- Permissions

11. Function Name : **stat_file()**

Input Parameters: FileName

Return Value: statistical information of file.

Description: This gives the Statistical information about the file & Displays Complete metadata of the specified file.

Information Displayed:

- File Name.
- Inode Number.
- File Size

- Actual File size
- Reference Count
- File Permission
- File Type

12. Function Name : **unlink_file()**

Input Parameters: FileName

Return Value: Exit status of function

Description: Deletes the specified virtual file.

Operation Performed:

- Removes the inode information.
- Frees the allocated memory.
- Updates the Super Block.
- Releases the file buffer.

13. Function Name : **write_file()**

Input Parameters: File Descriptor, Data that we want to write, Size of data.

Return Value: Number of bytes successfully written.

Description: Writes user-entered data into the selected virtual file. This writes data to file with FD-3.

Operation Performed:

- Checks write permission.
- Copies data into the file buffer.
- Updates Write Offset.
- Updates Actual File Size.

14. Function Name : **read_file()**

Input Parameters: File Descriptor, Address of empty Buffer, Size of data.

Return Value: Number of bytes successfully read.

Description: Reads the specified number of bytes from the virtual file.

Operation Performed:

- Checks read permission.
- Reads data from the current Read Offset.
- Updates the Read Offset.
- Returns the requested data.

15. Function Name : **open_file()**

Input Parameters: FileName, Mode of file

Return Value: File Descriptor

Description: Reads the specified number of bytes from the virtual file.

Operation Performed:

- Checks read permission.
- Reads data from the current Read Offset.
- Updates the Read Offset.
- Returns the requested data.

16. Function Name : **close_file()**

Input Parameters: FD of file name

Return Value: Exit status of function

Description: Closes the specified file descriptor.

Operation Performed:

- Releases the File Table.
- Decrements the Reference Count.
- Frees the File Descriptor.

17. Function Name : **Display_File()**

Input Parameters: File Name

Return Value: File content

Description: Displays the complete contents of the specified virtual file.

Operation Performed:

- Searches the file.
- Checks read permission.
- Prints the file contents stored in the virtual buffer.

18. Function Name : **Backup_File()**

Input Parameters: NA

Return Value: Backup files created during execution in folder

Description: It is used to Backup the files created Virtually during execution into HDD (RAM to HDD). This will automatically called before exit.

Operation Performed:

- Creates the fullpath with file name.
- Write the contents of file from buffer.

19. Function Name : **main()**

Input Parameters: NA

Return Value: NA

Description: Entry Point function of the CVFS project

Purpose :

The main() function serves as the entry point of the Customized Virtual File System (CVFS). It is responsible for initializing the Virtual File System, displaying the command prompt, accepting user commands, parsing the input, invoking the appropriate file operation functions, handling errors, and terminating the application safely.

The main() function acts as the Command Line Interface (CLI) of the project, allowing users to interact with the Virtual File System through various commands such as create, open, read, write, close, ls, stat, unlink, and exit.

Flow of Execution:

Step 1: Initialize the Virtual File System

At the beginning of execution, the following initialization functions are called:

- Initializes the Boot Block.
- Initializes the Super Block.
- Creates the Disk Inode List Block (DILB).
- Initializes the User File Descriptor Table (UFDT).

These functions prepare the Virtual File System before accepting any user commands.

Step 2: Display Welcome Message

The program displays a welcome message and informs the user that the Customized Virtual File System has started successfully.

Example:

```
C:\Users\ADMIN\Desktop\LB\CVFS>myexe
Booting process of Marvellous CVFS is completed

Marvellous CVFS : UAREA gets initalised succesfully
Marvellous CVFS : Super Block gets initalised succesfully
Marvellous CVFS : DILB gets created succesfully
-----
----- Marvellous CVFS Started Successfully -----
-----
Marvellous CVFS : >
```

Type 'help' to display all available commands.

Step 3: Infinite Command Processing Loop

The main() function continuously executes an infinite while loop until the user enters the exit command.

Inside this loop, the program:

- Displays the command prompt.
- Accepts the command entered by the user.
- Parses the command into individual tokens.
- Determines which command has been entered.
- Calls the corresponding function.
- Displays the result or error message.

Example:

Marvellous VFS : >

Step 4: Read User Input

The program accepts an entire command line from the user using input function- fgets().

Example commands:

```
create Demo.txt 3
open Demo.txt 3
write Demo.txt
read 3 10
display Demo.txt
ls
exit
```

The entered command is stored in a character array.

Step 5: Parse the Command

The command is divided into separate words using string parsing functions such as sscanf().

For example:

Input: create Demo.txt 3
is separated as:

```
Command[0] create
Command[1] Demo.txt
Command[2] 3
```

The total number of parameters is also counted to validate the command syntax.

Step 6: Execute the Appropriate Command

The first token of the command is compared with supported command names using conditional statements.

Depending on the command entered, the corresponding function is called.

Examples:

- create → Creates a new virtual file.
- open → Opens an existing file.
- write → Writes data into a file.
- read → Reads data from a file.
- display → Displays file contents.
- close → Closes an opened file.
- ls → Lists all files.
- stat → Displays file information.
- unlink → Deletes a virtual file.
- help → Displays all supported commands.
- man → Displays detailed information about a command.
- exit → Terminates the application.

Each command is validated before execution, ensuring that the required number of parameters is supplied.

Step 7: Error Handling

After executing each command, the returned status code is checked.

If the operation is successful, an appropriate success message is displayed.

If any error occurs, the corresponding error message is displayed.

Examples:

- File already exists.
- File not found.
- Permission denied.
- Invalid file descriptor.
- Insufficient memory.
- Invalid command.

This improves the usability and robustness of the application.

Step 8: Exit the Application

When the user enters the exit command, the following operations are performed:

- Closes all opened files.
- Creates a backup of all virtual files as backup functionality is called there.
- Frees all dynamically allocated memory.
- Releases system resources.
- Displays a termination message.
- Exits the program safely.
- Responsibilities of the Main Function

The main() function performs the following responsibilities:

1. Initializes all file system components.
2. Displays the command prompt.
3. Accepts commands from the user.
4. Parses command-line arguments.
5. Validates user input.
6. Calls the appropriate file operation functions.
7. Handles success and error messages.
8. Continues execution until the user terminates the application.
9. Performs cleanup before exiting.
10. Advantages of the Main Function Design
11. Provides a user-friendly command-line interface.
12. Separates user interaction from file operation logic.
13. Makes the program modular and easy to maintain.
14. Simplifies the addition of new commands in the future.
15. Ensures proper initialization and resource management.
16. Mimics the behavior of a real operating system shell by continuously accepting and executing commands.

Screenshot of output:

Bootng & Initialization of CVFS System:

```
C:\Users\ADMIN\Desktop\LB\CVFS>gcc CVFS_backup.c -o myexe

C:\Users\ADMIN\Desktop\LB\CVFS>myexe
Bootng process of Marvellous CVFS is completed

Marvellous CVFS : UAREA gets initialised succesfully
Marvellous CVFS : Super Block gets initialised succesfully
Marvellous CVFS : DILB gets created succesfully
-----
----- Marvellous CVFS Started Successfully -----
-----

Marvellous CVFS : >
```

Creat command:

Creates a new virtual file in the file system with the specified name and permission.

Permission Values:

- 1 – Read
- 2 – Write
- 3 – Read & Write

Operation Performed:

- Checks for free inode.
- Verifies whether the file already exists.
- Allocates memory for the file buffer.
- Creates a File Table entry.
- Updates the Super Block.
- Returns a File Descriptor (FD).

```
Marvellous CVFS : > creat Demo.txt 3
File Successfully created with FD :3

Marvellous CVFS : > creat Hello.txt 3
File Successfully created with FD :4

Marvellous CVFS : > creat Sample.txt 3
File Successfully created with FD :5
```

ls command:

Displays the names of all existing files.

Operation Performed:

- Traverses the inode list.
- Displays all files having a valid file type.

```
Marvellous CVFS : > ls
```

```
----- Marvellous CVFS Files Information -----
```

```
Demo.txt  
Hello.txt  
Sample.txt
```

Ls -a command:

Displays complete information about all files present in the Virtual File System.

Information Displayed:

- File Name.
- Inode Number.
- Actual File size
- Permissions

```
Marvellous CVFS : > ls -a
```

```
----- Marvellous CVFS Files Information -----
```

File Name	Inode Number	ActualFileSize	Permission
Demo.txt	1	0	3
Hello.txt	2	0	3
Sample.txt	3	0	3

Help Option:

Displays the list of all supported commands along with a brief description of each command.

```
Marvellous CVFS : > help
```

```
----- Marvellous CVFS Help Page -----
```

```
man :    It is used to display the manual page
clear :  It is used to clear the terminal screen
creat :  It is used to create new regular file
open :   It is used to open regular file
close :  It is used to close regular file
write :  It is used to write the data into the file
read :   It is used to read the data from the file
stat :   It is used to display statistical information of file
unlink : It is used to delete the file
exit :   It is used to terminate Marvellous CVFS
```

Man page option:

Displays detailed information, syntax, parameters, and usage of the specified command.

```
Marvellous CVFS : > man open
```

```
About : It is used to open the file
```

```
Usage : open File_Name Mode
```

```
File_Name : Name of file which needs to be open
```

```
Mode : mode of file for which we are opening that file
```

```
Marvellous CVFS : > man exit
```

```
About : It is used to Terminate the project
```

```
Usage : exit
```

```
Marvellous CVFS : > man creat
```

```
About : It is used to create new file
```

```
Usage : creat File_Name Permission
```

```
File_Name : Name of file that we want to create
```

```
Permission : Permission of new file
```

```
Permission : Read -> 1
```

```
Permission : Write -> 2
```

```
Permission : Read + Write -> 3
```

Stat Command:

```
Marvellous CVFS : > stat Demo.txt
```

This gives the Statistical information about the file & Displays complete metadata of the specified file.

Information Displayed:

- File Name.

- Inode Number.
- File Size
- Actual File size
- Reference Count
- File Permission
- File Type

```
Marvellous CVFS : > stat Demo.txt
```

```
-----Statistical Information of File-----
```

```
File Name : Demo.txt
Inode Number : 1
File Size : 1024
Actual File Size : 0
Reference Count : 1
File Permission : Read + Write
File Type: Regular File
-----
```

clear Command:

Marvellous CVFS : > clear

Clears the command prompt screen, making it easier to continue working with the Virtual File System.

write Command:

Marvellous CVFS : > write 3

Writes user-entered data into the selected virtual file. This writes data to file with FD-3.

Operation Performed:

- Checks write permission.
- Copies data into the file buffer.
- Updates Write Offset.
- Updates Actual File Size.

```
Marvellous CVFS : > write 3
Enter the Data that you want to write into the file...
Hello World! This is CVFS Project
33 bytes gets successfully written into file
```

Read command:

Marvellous CVFS : > read 3 10

Reads the specified number of bytes from the virtual file.

It reads 10 bytes from file with FD 3.

Operation Performed:

- Checks read permission.
- Reads data from the current Read Offset.
- Updates the Read Offset.
- Returns the requested data.

```
Marvellous CVFS : > read 3 10
Read operation is Successful..
Data from file is :
Hello Worl
```

```
Marvellous CVFS : > read 3 10
Read operation is Successful..
Data from file is :
d! This is
```

```
Marvellous CVFS : > read 3 10
Read operation is Successful..
Data from file is :
CVFS Proj
```

```
Marvellous CVFS : > read 3 10
Read operation is Successful..
Data from file is :
ect
```

```
Marvellous CVFS : > read 3 10
Error : Insufficient Data
```

display command:

Marvellous CVFS : > display Demo.txt

Displays the complete contents of the specified virtual file.

Operation Performed:

- Searches the file.
- Checks read permission.
- Prints the file contents stored in the virtual buffer.

```
Marvellous CVFS : > display Demo.txt
```

```
-----  
File Name : Demo.txt  
-----
```

```
Hello World! This is CVFS Project
```

```
File displayed successfully...
```

```
Marvellous CVFS : >
```

open command:

Marvellous CVFS : > open Demo.txt 3

Opens an existing virtual file in the specified mode and returns a new File Descriptor.

Mode Values:

- 1 – Read
- 2 – Write
- 3 – Read & Write

Operation Performed:

- Searches the inode list.
- Verifies file permissions.
- Allocates a new File Table.
- Updates the Reference Count.
- Returns a unique File Descriptor.

```
Marvellous CVFS : > open Demo.txt 3
```

```
File opened successfully
```

```
File Descriptor : 6
```

close command:

Marvellous CVFS : > close Demo.txt 3

Closes the specified file descriptor.

Operation Performed:

- Releases the File Table.
- Decrements the Reference Count.
- Frees the File Descriptor.

```
Marvellous CVFS : > close 4
```

```
File closed successfully
```


Unlink command:

Marvellous CVFS : > unlink Hello.txt

Deletes the specified virtual file.

Operation Performed:

- Removes the inode information.
- Frees the allocated memory.
- Updates the Super Block.
- Releases the file buffer.

```
Marvellous CVFS : > unlink Hello.txt
```

```
Marvellous CVFS : > ls
```

```
----- Marvellous CVFS Files Information -----
```

```
Demo.txt
```

```
Sample.txt
```

exit command:

Marvellous CVFS : > exit

Terminates the Virtual File System application.

Operation Performed:

- Creates a backup of all virtual files (if backup functionality is enabled).
- Frees all dynamically allocated memory.
- Closes all open files.
- Exits the application safely.

Conclusion:

The **Customized Virtual File System (CVFS)** is a memory-based file management system developed in the C programming language to simulate the core functionalities of a real operating system's file system. The project demonstrates how operating systems internally manage files, inodes, file descriptors, permissions, and memory without interacting directly with the physical storage device. The system provides a command-line interface through which users can perform various file operations.

Overall, the Customized Virtual File System successfully demonstrates the internal working principles of a file system while providing a practical implementation of operating system concepts.